

Big-O and Complexity Notes

Big-O tells you how an algorithm's resource use grows as the input size n increases. It helps compare solutions before worrying about exact seconds on one particular computer.

Core concepts

1. What is Big-O?

Big-O notation describes the growth rate of an algorithm, usually its running time or extra memory, as the input becomes larger. It focuses on the dominant work and ignores constants. For example, $O(2n)$ is treated as $O(n)$.

2. What does $O(1)$ mean?

$O(1)$ means **constant time**: the work stays roughly the same even if the input grows.

Example: accessing `numbers[0]` or looking up a key in a Python dictionary on average.

3. What does $O(n)$ mean?

$O(n)$ means **linear time**: if the input doubles, the work roughly doubles.

Example: looping through every element in a list once.

```
for number in numbers:
    print(number)
```

4. What does $O(n^2)$ mean?

$O(n^2)$ means **quadratic time**: if input size doubles, the amount of work becomes about four times larger. It commonly happens with a loop inside another loop.

```
for i in numbers:
    for j in numbers:
        print(i, j)
```

Binary search

5. Why is binary search $O(\log n)$?

Binary search checks the middle element of a sorted list and discards half of the remaining elements after every comparison.

For 16 elements, it can reduce the search area like this:

```
16 → 8 → 4 → 2 → 1
```

Because the list is repeatedly divided by 2, the number of steps grows logarithmically, not linearly. Binary search requires sorted data.

Source: [Algorithmic Complexity Notes](#)

Time vs space

6. What is the difference between time complexity and space complexity?

- **Time complexity:** How the number of operations grows with input size.
- **Space complexity:** How much additional memory the algorithm needs as input size grows.

Example: storing every element in a dictionary while scanning a list uses $O(n)$ extra space. If you only keep two index variables, it uses $O(1)$ extra space.

Dictionary and Two Sum

7. Why can a dictionary make Two Sum faster?

In the normal nested-loop method, you compare each number with many other numbers, which can take $O(n^2)$.

With a dictionary, for each number you calculate the needed partner and check whether it was already seen. Dictionary lookup is $O(1)$ on average, so the entire scan becomes $O(n)$ on average.

```
def two_sum(numbers, target):
    seen = {}

    for index, number in enumerate(numbers):
        needed = target - number

        if needed in seen:
            return [seen[needed], index]

    seen[number] = index
```

This improves speed but uses $O(n)$ additional memory for seen. Python dictionaries support average-case key lookup in $O(1)$.

8. What is the time complexity of dictionary lookup on average?

Dictionary lookup, such as `key in my_dict` or `my_dict[key]`, is $O(1)$ on average because dictionaries use hashing. In uncommon worst-case collision scenarios, it can degrade to $O(n)$.

Source: [Python Wiki: Time Complexity](#).

Two pointers

9. What is the two-pointer technique?

It is a problem-solving method where you keep two positions in an array or string and move them according to a condition.

A common example is finding two numbers in a sorted list that add up to a target:

```
def has_pair(numbers, target):
    left = 0
    right = len(numbers) - 1

    while left < right:
        total = numbers[left] + numbers[right]

        if total == target:
            return True
        elif total < target:
            left += 1
        else:
            right -= 1

    return False
```

If the sum is too small, move `left` right to increase it. If the sum is too large, move `right` left to decrease it. Each pointer moves at most n times, so this is usually $O(n)$ time and $O(1)$ extra space.

Python list insertion

10. Why is inserting at index 0 of a Python list usually $O(n)$?

Python lists behave like dynamic arrays. When you insert at the beginning:

```
numbers.insert(0, 99)
```

Python must shift every existing element one position to the right to make space for 99. For a list of n elements, up to n elements may move, so the operation is $O(n)$. Python's complexity reference lists `insert` as $O(n)$.

Source: [Python Wiki: Time Complexity](#).

